

Technical / Product Spec

Overview

Problem Statement: Dor needs one WhatsApp surface that can draft context-based replies for active threads and surface follow-up reminders for dormant threads without requiring a separate app.

Product Goal: Build a shared conversation ledger that stores inbound and outbound messages per client, then trigger reply drafting when a new message arrives and follow-up drafting when the x-time window has passed since the last outbound contact.

Recommended Shape: Use three internal capabilities: a conversation ledger, a reply generator, and a follow-up reminder generator.

Implementation Notes: Keep the WhatsApp bot as the only user-facing surface, store full conversation history for later context, and keep all generated drafts copy-ready and manual-send only.

Goals

- Store inbound and outbound WhatsApp messages in a normalized ledger per client and thread.
- Trigger a context-based reply draft whenever a new inbound message arrives.
- Trigger a follow-up reminder inside the WhatsApp bot when x time has passed since the last outbound message.
- Support export and re-upload of full conversation history when a thread needs more context than the default summary window.
- Keep Dor in control of sending every message manually.

Architecture

- WhatsApp webhook ingestion receives inbound messages and writes them to the ledger.
- Conversation storage keeps the raw messages, thread summary, and reminder metadata in a shared data model.
- Reply generation reads the latest thread context and drafts one suggested response for the current conversation.
- A weekly scheduler scans stored threads and identifies clients whose last outbound contact is older than the configured x-time threshold.

- The WhatsApp bot surfaces both reply drafts and follow-up reminders so Dor does not need a separate dashboard.
- An export/import path supports longer context windows for difficult or long-running threads.

Data model

- messages: message_id, client_id, thread_id, direction, sender_role, timestamp, body, source_channel, external_message_id.
- threads: thread_id, client_id, last_inbound_at, last_outbound_at, last_follow_up_due_at, message_count, summary.
- drafts: draft_id, client_id, thread_id, draft_type, source_message_ids, context_count, body, created_at, status.
- reminders: reminder_id, client_id, thread_id, due_at, last_checked_at, status, draft_id.
- exports: export_id, client_id, thread_id, exported_at, file_path, imported_at.

Automation

- On inbound webhook, persist the message, update the thread summary, and queue reply generation.
- On outbound manual send, persist Dor's reply and update last_outbound_at for the thread.
- A weekly job checks each client against the x-time threshold and identifies due follow-ups.
- When a thread is due, the reminder appears inside the WhatsApp bot together with a ready-made outreach message.
- If Dor uploads a full conversation export, the follow-up generator can rebuild the draft with the broader history.

Workflows

1. Inbound reply flow: a new WhatsApp message arrives, the ledger stores it, and the reply generator produces one suggested response from the latest thread context.
2. Follow-up reminder flow: the weekly scheduler finds overdue threads, then the WhatsApp bot shows Dor a ready-made outreach message for each due client.
3. Export/import flow: when a thread needs more context, Dor exports the full conversation, uploads it back into the follow-up flow, and regenerates the draft.

Implementation plan

1. Define the conversation ledger and draft record schemas.

2. Implement webhook ingestion and outbound message persistence.
3. Build the reply generator against stored thread history and summary data.
4. Add the weekly scheduler plus the reminder surface inside the WhatsApp bot.
5. Add export/import support and store the message count used for each draft.

Non-goals

- Do not send WhatsApp messages automatically on Dor's behalf.
- Do not create a separate reminder application outside the WhatsApp bot.
- Do not require Dor to manually assemble context from raw message logs.
- Do not use the full conversation by default for every draft.

Acceptance criteria

- A new inbound WhatsApp message is persisted and can trigger a reply draft.
- A weekly job identifies clients past the follow-up threshold and surfaces a ready-made message inside the WhatsApp bot.
- Both draft flows use stored history rather than only the latest message.
- The system can export the full conversation for deeper context and re-upload it into the drafting flow.
- Internal draft logs include source message ids and the number of messages used for context.

Risks

- Long threads may exceed the default context window and require summary trimming or export/import fallback.
- Timezone handling may cause reminders to fire early or late if thread timestamps are not normalized.
- Duplicate inbound webhooks could create duplicate message records unless message ids are de-duplicated.
- A stale summary could produce a weak draft if the thread is updated faster than the summary job runs.

Open questions

- What storage backend should hold the conversation ledger and generated draft records?

- Should the x-time threshold be global by default with per-client overrides, or configured per client only?
- Should the weekly scheduler generate all due drafts eagerly, or only when Dor opens the WhatsApp bot?
- What is the default context window for reply drafts before the full export fallback is needed?

Future ideas

- Add per-client reminder intervals and service-specific follow-up templates.
- Add automatic thread summaries to keep the reply context small on long conversations.
- Add draft analytics so Dor can see which reply and follow-up suggestions get used most often.